Does Word Order Matter for News Topic Classification?

A controlled comparison of bag-of-words, recurrent and attention models on AG News

Yusuf Bulut

November 2026 (revised edition of a course project)

Abstract

Do models that read word order classify news topics more accurately than models that ignore it? On the AG News benchmark, a mean-pooled embedding network (FNN), a vanilla RNN, an LSTM and an LSTM with attention are compared under shared hyperparameters, with random and GloVe-initialized embeddings, against a TF-IDF + logistic regression baseline. Each configuration is trained with three seeds across five training-set sizes (1k–108k articles) and four input lengths (8–64 tokens), 180 runs in total. On the full data, the GloVe FNN (92.02%), GloVe LSTM (92.21%), GloVe Attention LSTM (92.12%) and TF-IDF (92.04%) are within seed-to-seed noise of one another, while the GloVe LSTM models take about six times as much CPU training time as the GloVe FNN. With little data the order-blind models are clearly better: at 1,000 examples the GloVe FNN leads the GloVe LSTM by 2.8 points, and with random embeddings by 13.5. The first 8 tokens of an article alone give 85–88% accuracy. On this task, pretrained embeddings matter more than modeling word order.

1 Introduction

Recurrent networks and attention were designed to model word order, and they are standard tools for text classification. Topic classification, however, can often be solved from which words occur, regardless of their order. A story mentioning *touchdown* and *quarterback* is about sports wherever those words appear. Bag-of-words models are known to be strong on such tasks (Zhang et al. 2015; Joulin et al. 2017; Iyyer et al. 2015). This project asks how much modeling word order actually adds on a standard topic benchmark, and what it costs:

> On AG News, do models that read word order (RNN, LSTM, Attention LSTM) classify topics more accurately than an order-blind model that averages word embeddings, and does the answer depend on the amount of training data, the input length or the initialization of the embeddings?

The comparison is controlled: all neural models share their embedding size, hidden size, vocabulary, optimizer and stopping rule, and differ only in how they turn a sequence of word vectors into one document vector. A TF-IDF + logistic regression baseline anchors the results, and every configuration is repeated with three seeds so that differences can be compared with the noise.

2 Data

AG News (Zhang et al. 2015) contains news articles from four topics (World, Sports, Business, Sci/Tech), each consisting of a headline and a short description. The official split has 120,000 training and 7,600 test articles, balanced across classes. The dataset is loaded through Hugging Face `datasets` and is not redistributed with this project; its source describes it as provided for research and non-commercial use.

For every seed, the official training set is split 90/10 into training (108,000) and validation (12,000) sets, stratified by class. The test set is used only once per run, for the final evaluation of the weights with the best validation accuracy.

Preprocessing. AG News stores many HTML entities without their leading ampersand (`#39;s`, `quot;`), uses backslashes as line breaks, and occasionally contains escaped HTML tags. These are repaired before tokenization; the original version of this project skipped this step, which put tokens such as `#39;s` and `(reuters)` into the vocabulary. Text is then lowercased and split into alphanumeric tokens (dotted forms such as `u.s` are kept). After cleaning, articles have a median length of 38 tokens; 98% have 64 tokens or fewer, and none exceed 181.

3 Models

All neural models share the same embedding layer (vocabulary of the 20,000 most frequent training tokens appearing at least twice, 100 dimensions) and the same hidden size (128). They differ only in how a sequence of word vectors becomes a single document vector:

- **FNN (mean-pooled).** The masked mean of the word embeddings, followed by a one-hidden-layer MLP. This model is invariant to word order.
- **RNN.** A single-layer Elman RNN (Elman 1990); the hidden state at the last real (non-padding) token is classified.
- **LSTM.** As above with an LSTM cell (Hochreiter and Schmidhuber 1997).
- **Attention LSTM.** An LSTM whose states are pooled with additive attention (Bahdanau et al. 2015): a two-layer scorer assigns each position a weight, padding is masked, and the weighted sum of states is classified.

Each model is trained with randomly initialized embeddings and, separately, with embeddings initialized from GloVe 6B 100d (Pennington et al. 2014) and fine-tuned. GloVe covers 99% of the vocabulary.

Baseline. TF-IDF features over word unigrams and bigrams (sublinear term frequency, minimum document frequency 2) with multinomial logistic regression. The inverse regularization strength $C \in \{1, 4, 16\}$ is chosen on the validation set, the same data the neural models use for early stopping.

4 Experimental setup

Setting	Value
Optimizer	Adam, learning rate 10^{-3}, batch size 64, gradient-norm clipping at 1.0
Regularization	dropout 0.2 before the classifier
Stopping	up to 30 epochs; early stopping on validation accuracy, patience 3
Input length	first 64 tokens (main experiment)
Seeds	0, 1, 2. The seed controls the train/validation split, subsampling, initialization and batch order
Metrics	accuracy and macro F1 on all 7,600 test articles; mean $\pm$ standard deviation over seeds

Hyperparameters are shared by all models and were not tuned per model. Three experiments are run:

1. **Main comparison.** All models on the full training set, both embedding initializations.
2. **Learning curve.** Stratified training subsets of 1k, 3k, 10k and 30k articles, plus the full 108k, with the validation set held fixed.
3. **Input length.** Models trained and evaluated on only the first 8, 16, 32 or 64 tokens of each article (random-init embeddings).

All 180 runs are logged to a single results file from which every table and figure in this report is generated. Training was run on an Apple M2 laptop by three concurrent workers: LSTM-based models mostly on the MPS GPU backend and everything else on CPU (the device of every run is logged). Because the workers shared the machine, training cost is measured separately in a controlled benchmark.

5 Results

5.1 Main comparison

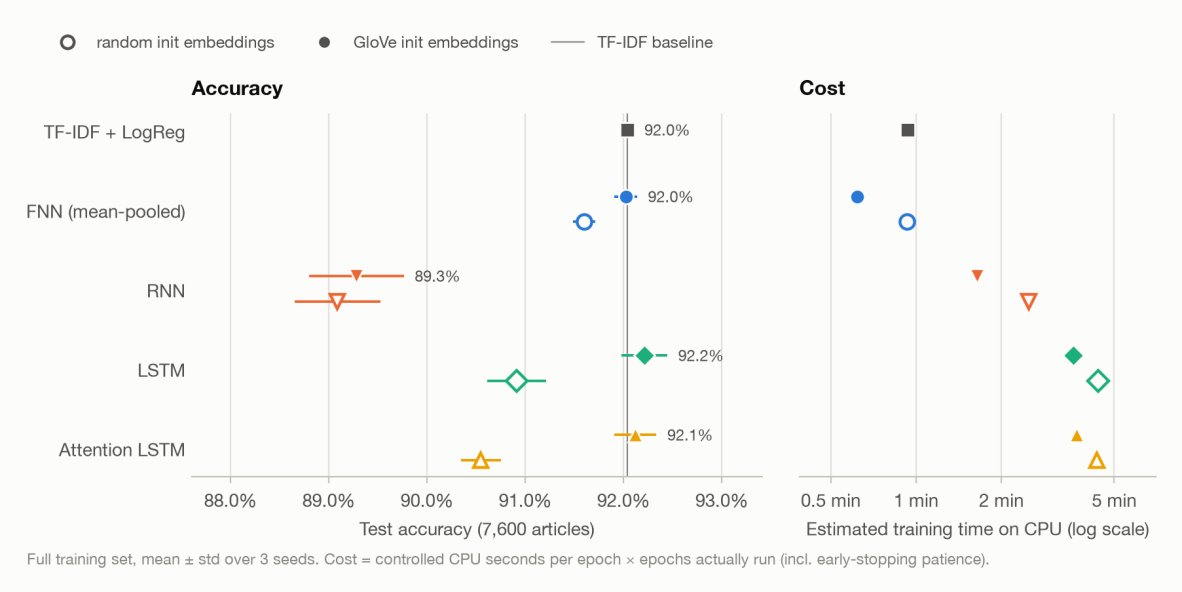


Figure 1: Test accuracy (left) and estimated CPU training time (right) on the full training set. Hollow markers: random-init embeddings; filled: GloVe-init. Error bars: $\pm$ 1 standard deviation over seeds.

With GloVe initialization, three neural models reach the accuracy of the TF-IDF baseline (92.04%): the order-blind FNN (92.02%), the LSTM (92.21%) and the Attention LSTM (92.12%). These differences are small relative to the noise. The standard error of a single accuracy measurement on 7,600 test articles is about 0.3 points, and the seed-to-seed standard deviation of the LSTM is 0.23. Because the seed fixes the train/validation split, the models can also be compared seed by seed against TF-IDF. The paired differences of the GloVe FNN (-0.21 to $+0.14$) and GloVe Attention LSTM (-0.09 to $+0.20$) change sign across seeds. Only the GloVe LSTM is ahead on two seeds out of three, and by at most 0.3 points.

The embedding initialization matters more than the architecture. GloVe improves the LSTM by 1.3 points and the Attention LSTM by 1.6, but the FNN by only 0.4. With random embeddings, the order-blind FNN (91.60%) outperforms the LSTM (90.91%) and the Attention LSTM (90.55%). A recurrent model has to learn word representations and how to compose them from the same labels, while a mean of embeddings only has to learn which words indicate which topic. Attention never improves on the plain LSTM, and the vanilla RNN trails every other model by about three points with either initialization.

Table 1: Main results (full training set, first 64 tokens). Mean ± standard deviation over three seeds. *Best epoch* is the epoch whose weights were kept. *CPU train time* is the controlled CPU time per epoch multiplied by the number of epochs actually run, including patience.

Model	Embeddings	Test accuracy (%)	Macro F1 (%)	Best epoch	CPU train time (min)	Params excl. embeddings
TF-IDF + LogReg	n/a	92.04 ± 0.07	92.02 ± 0.07	n/a	0.9	n/a
FNN (mean-pooled)	GloVe	92.02 ± 0.11	92.01 ± 0.11	4.3	0.6	13,444
FNN (mean-pooled)	random	91.60 ± 0.11	91.59 ± 0.11	8.0	0.9	13,444
RNN	GloVe	89.28 ± 0.48	89.26 ± 0.48	4.7	1.6	29,956
RNN	random	89.09 ± 0.44	89.07 ± 0.45	8.7	2.5	29,956
LSTM	GloVe	92.21 ± 0.23	92.19 ± 0.24	3.0	3.6	118,276
LSTM	random	90.91 ± 0.30	90.90 ± 0.29	4.3	4.4	118,276
Attention LSTM	GloVe	92.12 ± 0.22	92.11 ± 0.21	2.7	3.7	134,916
Attention LSTM	random	90.55 ± 0.21	90.53 ± 0.19	3.7	4.3	134,916

5.2 Learning curve

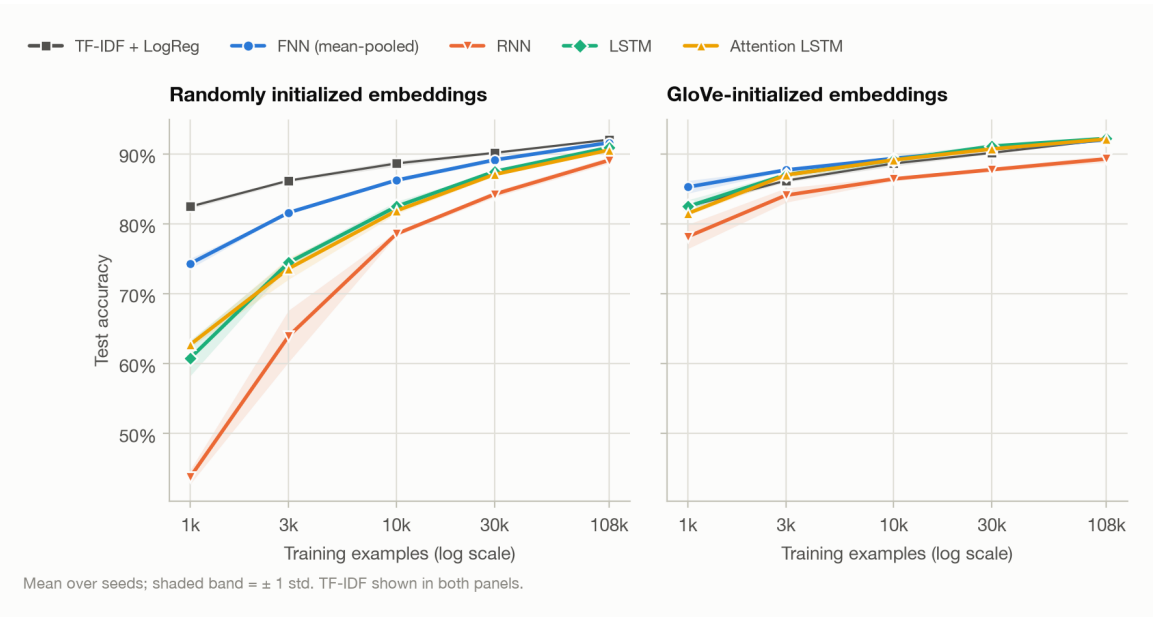


Figure 2: Test accuracy against training-set size (log scale) for random-init (left) and GloVe-init (right) embeddings. Shaded bands: ± 1 standard deviation over seeds.

With randomly initialized embeddings, every neural model trails TF-IDF at every training-set size, and the order-aware models trail the most. At 1,000 examples the FNN reaches 74.3%, the LSTM 60.8% and the RNN 43.8%, against 82.5% for TF-IDF. The gaps shrink as data grows. The LSTM's deficit to

the FNN falls from 13.5 points at 1,000 examples to 3.8 at 10,000 and 0.7 at 108,000, but it never closes. This fits the explanation from Section 5.1: a recurrent model must learn word representations and their composition from the same labels, so it needs more data, and within the 108,000 available articles the extra data does not make word order pay off.

GloVe initialization changes the small-data picture. The GloVe FNN beats TF-IDF on every seed up to 30,000 examples (by 2.0–3.3 points at 1,000 examples), and the GloVe LSTM and Attention LSTM are within one point of it from 3,000 examples on. The FNN is still ahead of the LSTM up to 10,000 examples, by 2.8 points on average at 1,000. At 30,000 examples the GloVe LSTM is ahead of the GloVe FNN on all three seeds, but by only 0.01–0.68 points, and on the full training set the sign of the difference varies with the seed. In this experiment, word order is worth at most a few tenths of a point, and only with pretrained embeddings and tens of thousands of labeled examples.

Table 2: Test accuracy (%) by number of training examples, mean over seeds.

Model	Embeddings	1,000	3,000	10,000	30,000	108,000 (all)
TF-IDF + LogReg	n/a	82.5	86.2	88.6	90.2	92.0
FNN (mean-pooled)	GloVe	85.3	87.7	89.3	90.7	92.0
FNN (mean-pooled)	random	74.3	81.6	86.2	89.1	91.6
RNN	GloVe	78.2	84.1	86.4	87.8	89.3
RNN	random	43.8	63.9	78.5	84.2	89.1
LSTM	GloVe	82.5	87.0	89.1	91.1	92.2
LSTM	random	60.8	74.5	82.5	87.5	90.9
Attention LSTM	GloVe	81.5	87.0	89.1	90.7	92.1
Attention LSTM	random	62.8	73.6	81.9	87.1	90.5

5.3 Input length

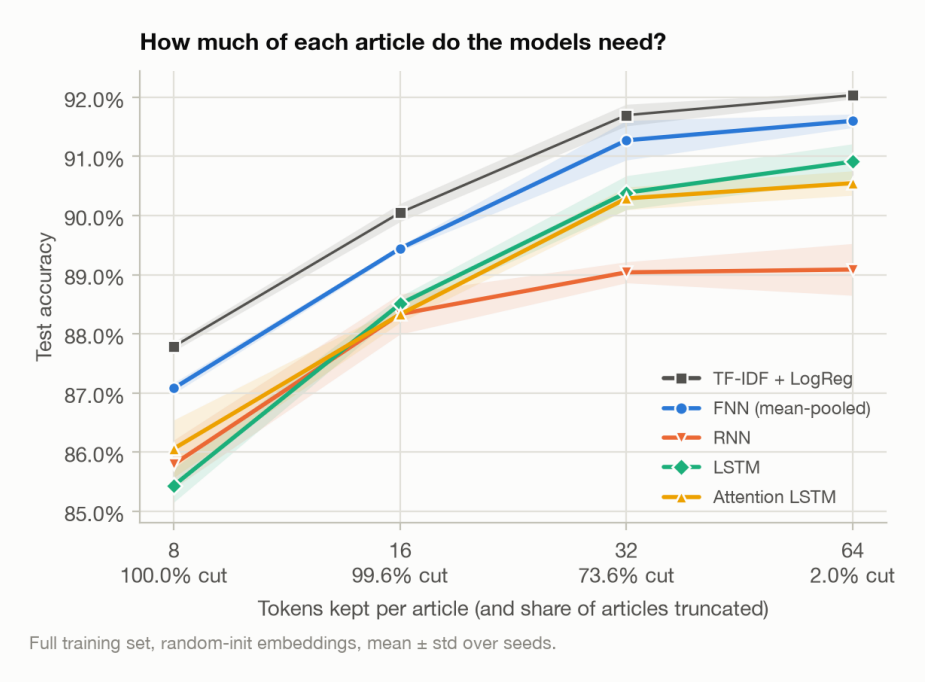


Figure 3: Test accuracy when each article is truncated to its first 8, 16, 32 or 64 tokens (random-init embeddings). Tick labels give the share of training articles that are cut at each length.

Table 3: Test accuracy (%) by number of tokens kept, mean over seeds.

Model	8 tokens	16 tokens	32 tokens	64 tokens
TF-IDF + LogReg	87.8	90.0	91.7	92.0
FNN (mean-pooled)	87.1	89.4	91.3	91.6
RNN	85.8	88.3	89.0	89.1
LSTM	85.4	88.5	90.4	90.9
Attention LSTM	86.1	88.3	90.3	90.5

Most of the signal sits at the start of the article. With only the first 8 tokens (roughly the headline), every model reaches 85–88% accuracy. Extending the input to 64 tokens adds between 3.3 points (RNN) and 5.5 points (LSTM). The order of the models is the same at every length: TF-IDF first, the FNN within a point of it, and the recurrent models behind. The LSTM gains the most from longer inputs, as a model that composes word order should, but it does not catch up with the order-blind FNN at any length. The vanilla RNN stops improving after 32 tokens (89.04% at 32 tokens, 89.09% at 64), even though 74% of articles are longer than 32 tokens. This is consistent with the known difficulty of carrying information through long sequences in a plain RNN (Hochreiter and Schmidhuber 1997).

5.4 Error analysis

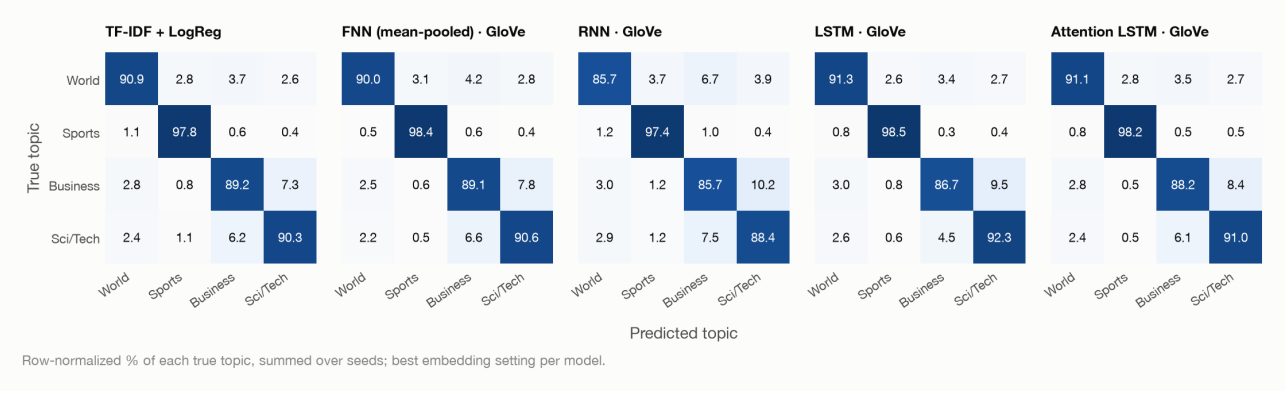


Figure 4: Row-normalized confusion matrices (percent of each true topic), summed over seeds, for the best embedding setting of each model.

Table 4: Per-class F1 (%), mean over seeds.

Model	Embeddings	World	Sports	Business	Sci/Tech
TF-IDF + LogReg	n/a	92.2	96.6	89.3	90.0
FNN (mean-pooled)	GloVe	92.2	97.1	88.8	89.9
FNN (mean-pooled)	random	91.8	96.9	88.3	89.3
RNN	GloVe	88.9	95.7	85.3	87.1
RNN	random	89.4	95.3	84.9	86.7
LSTM	GloVe	92.3	97.3	89.0	90.1
LSTM	random	91.3	96.1	87.8	88.5
Attention LSTM	GloVe	92.4	97.2	88.9	89.8
Attention LSTM	random	90.9	95.7	87.2	88.4

All models agree on which topics are hard. Sports is the easiest (F1 above 95% for every model) and Business the hardest, followed by Sci/Tech. For every model, confusions between Business and Sci/Tech make up 41–46% of all errors; an even split over the six possible pairs would give each about 17%. The next most frequent pair, World and Business, accounts for about 20%. Technology-company stories use the vocabulary of both topics (earnings, acquisitions, products), and some misclassified articles, such as the two at the bottom of Figure 5, could plausibly belong to either class. Among the strongest models, the per-class differences are a few tenths of a point. TF-IDF is slightly ahead on Business and slightly behind on Sports, so no architecture has a consistent advantage on any one topic.

5.5 What the attention layer attends to

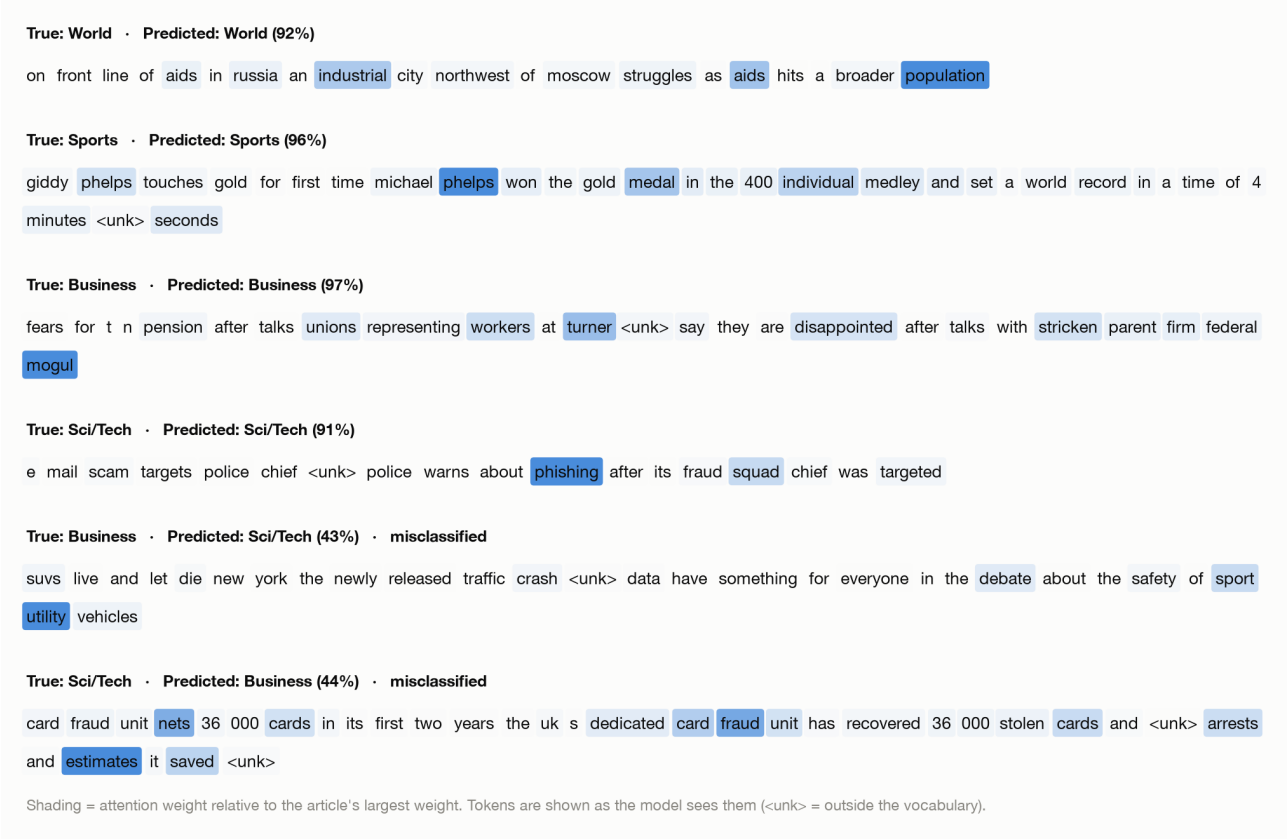


Figure 5: Six test articles shaded by the attention weights of the GloVe-initialized Attention LSTM (seed 0). The last two are misclassified.

The attention weights often land on topical words: *phelps* and *medal*, *phishing*, *fraud* and *cards*. Across the whole test set, however, the attention does not concentrate on the opening tokens, which roughly correspond to the headline. The first 8 tokens receive 19.3% of the attention mass on average and the last 8 tokens 24.7%, where uniform attention would give 22.4% to each. The random-init model leans further towards the end (15.2% and 24.8%; `results/attention_stats.txt`).

This does not mean the headline is uninformative: Section 5.3 shows that the first 8 tokens alone give 85–88% accuracy. An LSTM state at position t summarizes every token up to t, so a late state has already read the headline. Pooling late states is therefore an efficient way to read the whole article. The weights show where the model pools information, not which word caused a prediction (Jain and Wallace 2019; Wiegreffe and Pinter 2019).

5.6 Training cost

Table 5: Training time per epoch on the full training split (108,000 articles), measured one model at a time on an Apple M2.

Model	CPU s/epoch	MPS s/epoch
FNN (mean-pooled)	5.1	6.2
RNN	12.8	28.2
LSTM	35.9	18.6
Attention LSTM	39.1	21.1

TF-IDF + LogReg: 56 s for a full CPU fit (including the C search).

On the CPU, cost follows model complexity: an RNN epoch takes 2.5 times as long as an FNN epoch and an LSTM epoch 7 times as long. On the MPS backend, the RNN and the LSTM swap places. The LSTM runs about twice as fast as on the CPU and the RNN about twice as slow. The first version of this project timed its models on MPS and concluded that the RNN was slower than the LSTM, but that is a property of the backend's implementation, not of the models. The FNN, whose work is mostly an embedding lookup and a mean, gains nothing from MPS.

Multiplied by the number of epochs each model actually ran, training the GloVe FNN on the CPU takes about 0.6 minutes. That is less than the 0.9 minutes TF-IDF needs for vectorization and three logistic-regression fits during its search over C. The GloVe LSTM and Attention LSTM take 3.6 and 3.7 minutes (Table 1), about six times as long as the GloVe FNN, for at most 0.2 points of accuracy.

6 Discussion

Across all three experiments, the models that read word order never beat the best order-blind model by more than a few tenths of a point, and they come out ahead on average only with GloVe embeddings and at least 30,000 training examples. With random embeddings they are worse at every training-set size and every input length. Two properties of AG News explain why. First, the label is mostly determined by which words occur: the first 8 tokens already give 85–88% accuracy, and a bag of unigrams and bigrams matches every neural model. Second, the articles are short (median 38 tokens), so there is little long-range structure for a sequence model to exploit. The one place where length visibly matters, the plain RNN's plateau after 32 tokens, works against the order-aware model.

The results agree with earlier work showing that bag-of-words and averaged-embedding models are strong on topic classification (Iyyer et al. 2015; Joulin et al. 2017). They also show how much the conclusions depend on the amount of data. At 10,000 examples with random embeddings, the FNN leads the LSTM by 3.8 points; on the full data, the GloVe models are indistinguishable. A comparison run at a single data size, such as the 10,000 examples used in the first version of this project, can therefore report a gap that is specific to that size.

In practice, TF-IDF with logistic regression is the right first model for a topic task like this one. It fits in under a minute on a laptop CPU, and no neural model in this study beats it by a margin the test set can detect. When labeled data is scarce, a mean-pooled network over pretrained embeddings is the better choice, ahead of TF-IDF by 2.0–3.3 points at 1,000 examples. Recurrent and attention models would need a task in which word order changes the label to justify their cost, and AG News is not such a task.

7 Limitations

- **One dataset, one kind of task.** AG News is topic classification with strong keyword cues. The conclusions are not expected to carry over to tasks in which word order changes the meaning, such as negation, sentiment or entailment.
- **Shared hyperparameters.** All models use one configuration that was not tuned per model. A per-model search could change the gaps, although the gaps among the strongest models are within a few tenths of a point.
- **Three seeds.** Three seeds are enough to show that differences of a few tenths of a point are not stable, but not enough to estimate small differences precisely.
- **No pretrained Transformer.** Fine-tuned BERT-style models reach about 95% on AG News in published results and are the natural next comparison.
- **Attention as a diagnostic.** The attention weights apply to LSTM states that already mix in earlier tokens, so Section 5.5 describes where the model pools information, not why it made a decision.
- **Shared hardware.** Training runs shared one laptop, so the times logged during training are not comparable across runs. The cost comparison relies on the separate benchmark in Section 5.6.

8 Conclusion

On AG News, reading word order does not make a topic classifier meaningfully more accurate. An order-blind average of GloVe embeddings and a TF-IDF + logistic regression model reach 92.0% test accuracy, within noise of the best LSTM (92.2 $\pm$ 0.2%), at a fraction of the training cost, and the order-blind models are clearly better when labeled data is scarce. Pretrained embeddings matter far more than the choice of architecture, especially for the recurrent models.

The main lesson of the revision concerns method rather than models. The original version used about a quarter of the test set and a single seed, and ran a sequence-length experiment whose two longest conditions saw identical inputs. Under those conditions, differences of 1–3 points looked like clear findings. With the full test set, three seeds and paired comparisons, the differences among the strongest models are within the noise.

Reproducibility

Code, results and instructions are at https://github.com/Yusufb7841/word-order-text-classification. `make results` reruns every experiment, regenerates every figure and table, and rebuilds this PDF. Unit tests run in continuous integration on every push.

Changes from the original course version

The first version of this project was a course submission built in about three hours. A later review found several problems serious enough to change its conclusions, and this edition was rebuilt to fix them:

- **Data.** v1 trained on 10,000 of the 120,000 training articles and tested on 2,000 of the 7,600 test articles. All data is now used, and the effect of training-set size is measured explicitly.
- **Statistical support.** v1 reported a single seed. With 2,000 test articles, the 95% confidence interval of an accuracy near 82% is about $\pm$ 1.7 points, so several of its "clear" differences were within noise. All results are now means over three seeds, reported with standard deviations.

- **A sequence-length experiment that could not show anything.** v1 compared maximum lengths of 30, 80 and 150 tokens. Only 0.5% of articles exceed 80 whitespace tokens and none exceed 150, so the 80 and 150 conditions saw identical inputs. The new lengths (8–64) were chosen from the measured length distribution.
- **Tokenization.** Whitespace splitting left HTML-entity fragments and punctuation attached to words (`#39;s`, `quot;`, `(reuters)`, `said.`).
- **Inconsistent artifacts.** v1's RNN confusion matrix came from a different run than its reported metrics (63.8% vs 67.8% accuracy), and the data behind its sequence-length plot was missing. Every number is now generated from a single results file.
- **Cost measurement.** v1 reported the RNN as slower than the LSTM. That turned out to be a property of PyTorch's MPS backend (Section 5.6), not of the models. v1 also compared parameter counts that were 99% embedding table.
- **Baselines.** v1 had no reference outside the four neural models. This edition adds TF-IDF with logistic regression, and GloVe initialization for every neural model.

AI assistance

I built version 1 as a course project with the assistance of AI tools (ChatGPT and OpenAI Codex), which helped with planning, coding, debugging and drafting parts of the report.

For this edition I went back to fix what the original got wrong and turn it into a rigorous experiment. I set the scope: the full dataset with three seeds, a learning curve, a corrected input-length test, a TF-IDF baseline, GloVe embeddings, attention visualizations, automated tests and a new report. Claude Code carried out the implementation: it audited the original code and results, restructured and tested the codebase, ran the experiments and drafted the text. I directed the work, reviewed the code and the write-up, and checked the reported numbers against the saved results. The conclusions, and any errors, are mine.

References

Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. 2015. "Neural Machine Translation by Jointly Learning to Align and Translate." *International Conference on Learning Representations*.

Elman, Jeffrey L. 1990. "Finding Structure in Time." *Cognitive Science* 14 (2): 179–211.

Hochreiter, Sepp, and Jürgen Schmidhuber. 1997. "Long Short-Term Memory." *Neural Computation* 9 (8): 1735–80.

Iyyer, Mohit, Varun Manjunatha, Jordan Boyd-Graber, and Hal Daumé III. 2015. "Deep Unordered Composition Rivals Syntactic Methods for Text Classification." *Proceedings of ACL-IJCNLP*, 1681–91.

Jain, Sarthak, and Byron C. Wallace. 2019. "Attention Is Not Explanation." *Proceedings of NAACL-HLT*, 3543–56.

Joulin, Armand, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. 2017. "Bag of Tricks for Efficient Text Classification." *Proceedings of EACL*, 427–31.

Pennington, Jeffrey, Richard Socher, and Christopher D. Manning. 2014. “GloVe: Global Vectors for Word Representation.” *Proceedings of EMNLP*, 1532–43.

Wiegreffe, Sarah, and Yuval Pinter. 2019. “Attention Is Not Not Explanation.” *Proceedings of EMNLP-IJCNLP*, 11–20.

Zhang, Xiang, Junbo Zhao, and Yann LeCun. 2015. “Character-Level Convolutional Networks for Text Classification.” *Advances in Neural Information Processing Systems*.